

Course 2: LoRa Basics Modem

LBM Architecture & Integration

USP Training Series

LoRaWAN Development with Zephyr RTOS

November 21, 2025

Course Overview

What You'll Learn

- LoRa Basics Modem Architecture
 - Event-driven programming model
 - Modem API and services
 - Stack lifecycle management
- Advanced Features
 - Low power optimization
 - GNSS geolocation
 - LoRaWAN Relay
- Hands-On Labs (5 labs)
 - Lab 2.1: Build and run periodical uplink
 - Lab 2.2: Custom event handler
 - Lab 2.3: Low power configuration
 - Lab 2.4: GNSS geolocation
 - Lab 2.5: Relay TX configuration

What is LoRa Basics Modem?

Semtech's Official LoRaWAN Stack

Definition

LoRa Basics Modem (LBM) is Semtech's production-ready LoRaWAN stack implementing the complete MAC layer protocol.

Features:

- LoRaWAN 1.0.4 certified
- Event-driven architecture
- Low power optimized
- Built-in services:
 - GNSS scanning
 - LoRaWAN Relay
 - FUOTA support
 - Class A/B/C

Version Info:

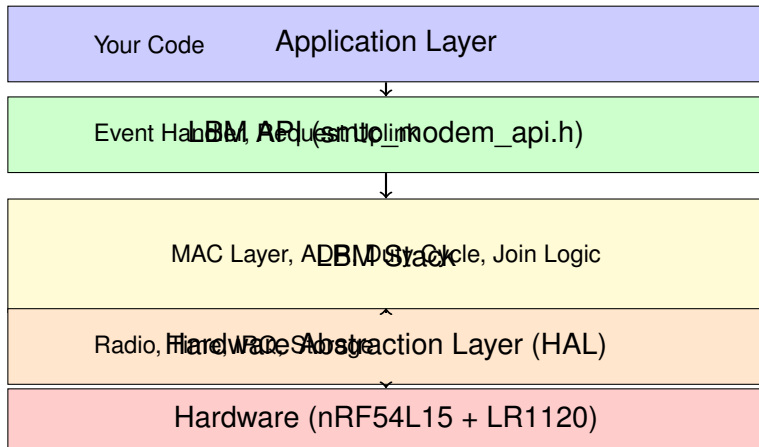
- Current: v4.9.0
- License: BSD-3-Clause
- Maintained by Semtech
- Used in production

USP Integration:

- Zephyr RTOS port
- Device tree config
- West workspace

LBM Architecture

Software Stack Layers



Key Point: Application code only interacts with LBM API, not hardware directly.

Event-Driven Programming

How LBM Communicates with Application

Event Loop Pattern

LBM uses events to notify the application of state changes and incoming data.

```
void event_handler(void)
{
    smtc_modem_event_t event;
    uint8_t pending_count;

    while (smtc_modem_get_event(&event, &pending_count) == SMTC_MODEM_RC_OK) {
        switch (event.event_type) {
            case SMTC_MODEM_EVENT_JOINED:
                LOG_INF("Network joined!");
                break;

            case SMTC_MODEM_EVENT_TXDONE:
                LOG_INF("TX done: %u", event.event_data.txdone.status);
                break;

            case SMTC_MODEM_EVENT_DOWNDATA:
                LOG_INF("Downlink received");
                handle_downlink(&event);
                break;
        }
    }
}
```

LBM Event Types

Complete Event Reference

Event	Description
JOINED	Network join successful
JOINFAIL	Join attempt failed
TXDONE	Uplink transmission complete
DOWNDATA	Downlink data received
LINK_CHECK	Link check result available
CLASS_B_STATUS	Class B beacon status
ALARM	Alarm timer expired
GNSS_SCAN_DONE	GNSS scan completed
RELAY_TX_SYNC	Relay TX synchronized
FUOTA	Firmware update event

Best Practice: Process events in a dedicated thread to avoid blocking modem engine.

LBM API Basics

Common Operations

Initialization:

```
smtc_modem_init(event_handler);  
smtc_modem_set_region(STACK_ID, SMTC_MODEM_REGION_EU_868);
```

Join Network:

```
smtc_modem_join_network(STACK_ID);
```

Send Uplink:

```
uint8_t payload[] = {0x01, 0x02, 0x03};  
smtc_modem_request_uplink(STACK_ID,  
                           2,          // Port  
                           false,     // Unconfirmed  
                           payload,  
                           3);        // Length
```

Get Status:

```
smtc_modem_status_mask_t status;  
smtc_modem_get_status(STACK_ID, &status);  
if (status & SMTC_MODEM_STATUS_JOINED) {  
    LOG_INF("Connected to network");  
}
```

Power Consumption Breakdown

Where Does Power Go?

State	Current	Duration	Energy
Sleep	25 μ A	95%	0.57 mAh/day
TX (SF7, +14dBm)	100 mA	41 ms $\times$ 24	0.027 mAh/day
RX1 Window	15 mA	100 ms $\times$ 24	0.010 mAh/day
RX2 Window	15 mA	100 ms $\times$ 24	0.010 mAh/day
Total (60s interval):			0.62 mAh/day

Battery Life Calculation (2000 mAh):

$$\text{Life} = \frac{2000 \text{ mAh} \times 0.7}{0.62 \text{ mAh/day}} = 2258 \text{ days} \approx 6.2 \text{ years}$$

Key Insight

Sleep current dominates for long-lived sensors! Optimize sleep mode first.

Low Power Strategies

Extend Battery Life

1. Increase Uplink Interval

- 60s → 300s (5min): 5× battery life improvement
- Consider application requirements

2. Optimize Spreading Factor

- SF12: 991 ms ToA, but better sensitivity
- SF7: 41 ms ToA, but requires good link
- Use ADR to find optimal balance

3. Reduce Payload Size

- Fewer bytes = shorter ToA
- Use efficient encoding (binary, not JSON!)

4. Disable Unnecessary Features

- No LED blinking
- Disable UART console in production
- Use confirmed uplinks sparingly

5. Hardware Optimization

- Enable Zephyr power management

GNSS with LR1120/LR1121

Satellite-Based Positioning

How It Works:

- 1 LR1120 scans for GPS/BeiDou satellites
- 2 Generates NAV (navigation) message
- 3 Send NAV to LoRa Cloud via LoRaWAN
- 4 Cloud solver returns position

Advantages:

- No dedicated GPS receiver needed
- Passive scanning (no TX)
- Works with obstructed view

Accuracy:

- Outdoor, clear sky: 10-30m
- Near window: 30-100m
- Indoor: May fail or >100m

Scan Modes:

STATIC: Device not moving, longer scan

MOBILE: Device moving, shorter scan

Time to Fix:

- With assistance: 30-60s
- Cold start: 2-5 min

GNSS Implementation

Code Example

```
// Configure GNSS
smtc_modem_gnss_set_constellation(STACK_ID,
    SMTC_MODEM_GNSS_CONSTELLATION_GPS_BEIDOU);

// Set assistance position (approximate location)
smtc_modem_gnss_assistance_position_t assist = {
    .latitude = 37.7749,
    .longitude = -122.4194,
};
smtc_modem_gnss_set_assistance_position(STACK_ID, &assist);

// Start scan
smtc_modem_gnss_scan(STACK_ID, SMTC_MODEM_GNSS_MODE_STATIC);

// In event handler:
case SMTC_MODEM_EVENT_GNSS_SCAN_DONE:
    uint8_t nav_message[255];
    uint16_t nav_size;

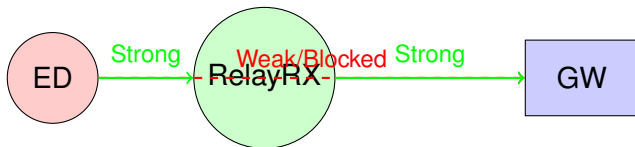
    smtc_modem_gnss_get_event_data_scan_done(STACK_ID,
                                              nav_message,
                                              &nav_size);

    // Send NAV to cloud solver
    smtc_modem_request_uplink(STACK_ID, 192, false,
                              nav_message, nav_size);

    break;
```

LoRaWAN Relay

Extend Network Coverage



Use Cases:

- Indoor sensors with outdoor gateway
- Extending range in difficult terrain
- Connecting devices in basements/underground

Types:

Relay RX: Mains-powered, Class C, forwards uplinks

Relay TX: Battery-powered end device, relayed by RX

Relay TX Configuration

Enable Relay for End Device

```
// After join, enable Relay TX
smtc_modem_relay_tx_enable(STACK_ID,
    SMTC_MODEM_RELAY_TX_ACTIVATION_MODE_ENABLE);

// In event handler:
case SMTC_MODEM_EVENT_RELAY_TX_SYNC:
    LOG_INF("Synchronized with Relay RX");
    LOG_INF("  Relay DevAddr: 0x%08X",
        event.event_data.relay_tx_sync.relay_dev_addr);
    break;

case SMTC_MODEM_EVENT_TXDONE:
    // Check if uplink was relayed
    smtc_modem_relay_tx_uplink_info_t info;
    smtc_modem_relay_tx_get_last_uplink_info(STACK_ID, &info);

    if (info.was_relayed) {
        LOG_INF("Uplink was RELAYED");
        LOG_INF("  Via: 0x%08X", info.relay_dev_addr);
    } else {
        LOG_INF("Direct uplink (not relayed)");
    }
    break;
```

Power Benefit: Relay TX can use lower TX power, saving battery!

Lab 2.1: Build and Run Periodical Uplink

Your First LBM Application

Objective: Build, flash, and run the LBM periodical uplink sample.

Steps:

- 1 Configure credentials in device tree overlay
- 2 Build: `west build -p -b xiao_nrf54l15_nrf54l15_cpuapp \`
`-shield semtech_lr1120mb1dis`
`samples/usp/lbm/periodical_uplink`
- 3 Flash: `west flash`
- 4 Monitor: `minicom -D /dev/ttyACM0 -b 115200`
- 5 Verify on network server (TTN/ChirpStack)

Expected Output:

- Join successful within 10 seconds
- Uplinks every 60 seconds
- RSSI/SNR values logged

Deliverables:

- Serial log showing join + 10 uplinks
- Network server screenshot

Lab 2.2: Custom Event Handler

Advanced Event Processing

Objective: Implement custom logic to track success rate and detect failures.

Features to Implement:

- 1 Track uplink success rate (confirmed vs total)
- 2 Alert if 3 consecutive failures
- 3 Log RSSI/SNR of downlinks
- 4 Automatic ADR profile switching based on link quality

Example Alert Logic:

```
static uint8_t consecutive_failures = 0;

case SMTC_MODEM_EVENT_TXDONE:
    if (event.event_data.txdone.status == SMTC_MODEM_EVENT_TXDONE_CONFIRMED) {
        consecutive_failures = 0;
    } else {
        consecutive_failures++;
        if (consecutive_failures >= 3) {
            send_alert_uplink(); // Notify server
        }
    }
    break;
```

Deliverables:

Lab 2.3: Low Power Configuration

Optimize for Battery Life

Objective: Configure LBM for minimum power consumption.

Tasks:

- 1 Build with low power config: `prj_lowpower.conf`
- 2 Measure power with Nordic Power Profiler Kit II
- 3 Compare different uplink intervals (60s, 300s, 900s)
- 4 Calculate battery life for each configuration

Measurements to Take:

- Sleep current (should be $< 30 \mu\text{A}$)
- TX current and duration
- RX window current and duration
- Average current per uplink cycle

Expected Results:

Interval	Avg Current	Battery Life
60 seconds	500 μA	166 days
300 seconds	100 μA	833 days
900 seconds	45 μA	1,851 days

Lab 2.4: GNSS Geolocation

Satellite-Based Positioning

Objective: Implement GNSS scanning and solve position with LoRa Cloud.

Prerequisites:

- LR1120 or LR1121 radio (has GNSS capability)
- Clear view of sky (outdoors or near window)
- LoRa Cloud account with MGMS token

Steps:

- 1 Configure GNSS constellation (GPS + BeiDou)
- 2 Set assistance position (approximate location)
- 3 Start GNSS scan after join
- 4 Send NAV message to LoRa Cloud
- 5 Receive solved position as downlink
- 6 Visualize on map (Python script provided)

Deliverables:

- Serial log showing NAV message and solved position
- HTML map with position marker

Lab 2.5: Relay TX Configuration

Extended Range with Relay

Objective: Configure device as Relay TX (relayed end-device).

Prerequisites:

- Two devices: one Relay TX (this lab), one Relay RX
- Network server supporting LoRaWAN Relay

Setup:

- 1 Configure and deploy Relay RX near gateway
- 2 Configure this device as Relay TX
- 3 Enable Relay TX mode after join
- 4 Verify uplinks are relayed (check metadata)
- 5 Measure power savings from reduced TX power

Verification:

- Serial log shows "Relay TX synchronized"
- Uplink events indicate "was relayed"
- Network server shows relay metadata

Deliverables:

- Relay TX code

Course 2 Summary

Key Takeaways

You've Learned:

- LBM Architecture: Event-driven, modular, production-ready
- Event Processing: How to handle JOIN, TXDONE, DOWNDATA events
- Low Power: Optimize sleep, interval, SF, payload for max battery life
- GNSS: Satellite positioning without dedicated GPS receiver
- Relay: Extend coverage using relay RX/TX devices
- Practical Skills: Build, optimize, and deploy LBM applications

Best Practices:

- Always use event-driven pattern (don't poll!)
- Test power consumption early in development
- Use ADR for optimal SF selection
- Consider Relay for indoor/difficult deployments

Thank You!

Questions?

https://github.com/dracu2400-svg/usp_zephyr